Proyecto integrador: Pipeline de streaming para cadena de frío ultrabaja con Kafka y Apache Beam

Rudolf Niedhammer[1]

[1] Maestría en Inteligencia Artificial, Facultad Politécnica, Universidad Nacional de Asunción, Streaming de datos y sus aplicaciones · Docente: Rodrigo Parra, M.Sc.

Abstract. Pipeline de extremo a extremo compuesto por replay determinista, Apache Kafka, Apache Beam sobre Flink y un destino de almacenamiento idempotente en SQLite para el monitoreo térmico de un contenedor de vacunas a −80/−60 °C, empleando datos reales de un ensayo del Oak Ridge National Laboratory. Se implementan y verifican ventanas temporales por tiempo de evento, deduplicación con horizonte finito, incorporación de lecturas tardías, descarte por exceso de retraso (en Docker a velocidad real y con `TestStream`) y aislamiento de registros inválidos hacia una cola de mensajes no procesables (DLQ).

Keywords: Kafka · Apache Beam · Flink · cadena de frío · event time · watermark

1 Problema, usuarios y justificación de streaming

El traslado final de vacunas sensibles a la temperatura requiere mantener temperaturas de ultracongelación (−80 °C a −60 °C) mediante hielo seco para abastecer centros asistenciales sin infraestructura de refrigeración avanzada. Este trabajo aborda la detección continua de anomalías térmicas utilizando datos reales de un ensayo del Oak Ridge National Laboratory con Carrier [1].

El pipeline responde a una necesidad analítica específica: para cada caja contenedora (*pod*) y en ventanas fijas de 5 minutos, computar las temperaturas mínima, media y máxima, contabilizar lecturas válidas únicas y determinar si existió una desviación de rango admisible. Dos perfiles consumen esta información: el operador logístico, que requiere alertas tempranas para aplicar medidas correctivas en tránsito (como recargar hielo seco tras una apertura prolongada), y el auditor de calidad, que evalúa el historial por lote para certificar la estabilidad térmica antes de la entrega. La sublimación de refrigerante y la apertura accidental de compuertas provocan variaciones críticas en minutos; un procesamiento por lotes diario identificaría las anomalías de forma tardía, cuando el producto biológico ya habría sido administrado o descartado.

2 Arquitectura del sistema

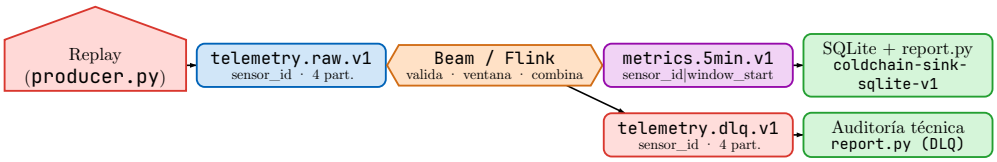


Fig. 1. Arquitectura lógica de procesamiento (flujo desacoplado; ver Tabla 1)

Tabla 1. Componentes del entorno de ejecución

Componente	Versión	Función principal
Apache Kafka (KRaft)	4.1.1	Ingesta de telemetría cruda, métricas agregadas y DLQ (un broker en entorno de laboratorio)
`kafka-init` / `dataset-init`	–	Servicios efímeros de inicialización: creación de tópicos y preparación de datos del ensayo
Apache Beam	2.74.0 (Python)	Validación de esquema, ventanas temporales, combinación incremental y emisión de paneles
Apache Flink	1.19.3	Motor de ejecución distribuido (`jobmanager` y 2 `taskmanagers`; `PortableRunner`, paralelismo 2)
`beam-job-server`	2.74.0	Servicio de traducción del grafo Beam al entorno de ejecución Flink
SQLite	Estándar	Almacenamiento final idempotente gestionado por el servicio consumidor (`data/out/metrics.sqlite`)

3 Contratos de eventos, tópicos, claves y particionado

El tópico de entrada `coldchain.telemetry.raw.v1` implementa el esquema versión 2:

```
{
  "schema_version": 2,
  "event_id": "ULT1-b07@20201219T043012Z",
  "event_time": "2026-09-25T14:03:12Z",
  "sensor_id": "ULT1-b07",
  "asset_id": "ULT1",
  "asset_type": "contenedor",
  "payload": {
    "temp_c": -80.63,
    "temp_f": -113.13,
    "source_time": "2020-12-19T04:30:12Z"
  }
}
```

El identificador `event_id` se genera de forma determinista combinando `{sensor_id}@{source_time}`, garantizando que las retransmisiones por fallas de red posean idéntico identificador y puedan ser deduplicadas sin depender del reloj de ingesta. La evolución del esquema admite retrocompatibilidad con esquemas previos de telemetría al volver opcionales atributos como humedad y nivel de batería, rechazando versiones no soportadas hacia la DLQ.

El tópico de salida `coldchain.metrics.5min.v1` (`schema_version: 1`, clave `aggregate_id` = `sensor_id|window_start`) emite: `sensor_id`, `asset_id`, `window_start`, `window_end`, `n_readings` (conteo de eventos únicos), `avg_temp_c`, `min_temp_c`, `max_temp_c`, `out_of_range`, `pane_timing`, `pane_index`, `is_final` y `emitted_at`.

Tabla 2. Topología de tópicos en Kafka

Tópico	Estrategia de clave	Part.	Contenido emitido
`coldchain.telemetry.raw.v1`	`sensor_id`	4	Lecturas brutas por sensor (`ULT1-b01` a `b20`)
`coldchain.metrics.5min.v1`	`sensor_id|window_start`	4	Paneles de agregación temporal por ventana de 5 min
`coldchain.telemetry.dlq.v1`	`sensor_id` o `unknown`	4	Eventos descartados con tipificación de causa

La clave de partición en la entrada es `sensor_id`. Esta elección preserva el orden de emisión por cada sensor físico y garantiza una dispersión equitativa del tráfico, ya que los 20 pods transmiten a intervalos idénticos de 10 segundos. Si se hubiese utilizado el identificador global del contenedor (`asset_id: "ULT1"`), todo el flujo se habría concentrado en una única partición, generando un desbalance total (*key skew*).

Para las 20 claves activas y un paralelismo de 2 en los nodos de cómputo, se definieron 4 particiones. El particionador por defecto de librdkafka distribuye las claves según el cálculo CRC32 en una proporción de 5, 6, 5 y 4 claves por partición. Esto arroja un desbalance máximo de 1,50× entre particiones (6 615 frente a 4 405 mensajes en el ensayo completo), manteniéndose por debajo del umbral de 2× establecido para justificar un particionador algorítmico a medida.

4 Pipeline en Apache Beam

Tabla 3. Fases secuenciales y transformaciones del pipeline

#	Transformación	Propósito operativo
1	`Read telemetry from Kafka`	Consumo con `ReadFromKafka` (política `CreateTime`), lectura clásica (`use_deprecated_read`) y compensación inicial `earliest`
2	`Parse and validate`	Validación contractual con enrutamiento de excepciones hacia la DLQ
3	`Assign event time`	Asignación de marcas de tiempo mediante `TimestampedValue` a partir del atributo `event_time`
4	`Fixed 5-min windows`	Segmentación en ventanas de 300 s con disparadores tempranos, tardíos y retraso permitido de 900 s
5	`Key by sensor` + `Combine stats`	Agregación incremental en `CombinePerKey` con deduplicación por clave integrada
6	`Format panes`	Estructuración de metadatos de ventana, pane y cálculo de identificador estable `aggregate_id`
7	`Drop empty panes`	Filtrado de paneles emitidos con `n_readings = 0` al cierre de lectura acotada
8	`Write metrics to Kafka`	Publicación en Kafka particionado por `aggregate_id`
9	`Reshuffle` + `Write DLQ`	Aislamiento de la etapa de captura y persistencia de eventos rechazados

5 Tiempo de evento, ventanas temporales, marca de agua y paneles

El orden temporal se rige exclusivamente por `event_time`, estructurado en ventanas fijas disjuntas de 5 minutos `[inicio, fin)`. La marca de agua (*watermark*) la calcula `KafkaIO` a partir del metadato `CreateTime`: cada partición registra el valor máximo observado, y la marca del pipeline asume el mínimo de las cuatro particiones. Debido a que múltiples sensores publican concurrentemente en cada partición, la marca de agua progresa de forma ininterrumpida aunque un sensor específico interrumpa su enlace.

La política de disparadores combina emisiones anticipadas (`early`) cada 5 segundos de tiempo de procesamiento, una emisión puntual (`on-time`) cuando la marca de agua rebasa el límite superior de la ventana, y emisiones tardías (`late`) por cada evento adicional recibido. El parámetro `allowed_lateness` se fijó en 15 minutos (900 segundos), margen suficiente para cubrir el percentil 99 de reconexión tras pérdidas de cobertura en ruta (supuesto de diseño: 9 minutos). La acumulación se configuró en modo `ACCUMULATING`.

5.1 Comportamiento en replay acelerado (20×)

La implementación de `CreateTime` en KafkaIO vincula la marca de agua al reloj del sistema: si el `event_time` del registro recibido es superior a la hora local, se trunca al instante presente; si la lectura está al día y el evento es anterior, se aproxima a la hora actual. En un replay acelerado a 20×, los intervalos de espera se reducen, pero los atributos `event_time` avanzan casi 3 horas por delante del reloj físico. Como consecuencia, la marca de agua queda rezagada respecto a los datos, los paneles se emiten como `EARLY` y las lecturas tardías del sensor `ULT1-b12` corrigen su ventana en lugar de ser descartadas.

Durante la experimentación, se evaluó utilizar la lectura moderna basada en SDF (Splittable DoFn), pero los puntos de control (*checkpoints*) de Flink expiraban de forma sistemática y la rama de la DLQ perdía el último bloque de registros. Por ello, se adoptó formalmente la lectura clásica con `--experiments=use_deprecated_read`, logrando puntos de control exitosos y la entrega íntegra de los registros malformados.

5.2 Verificación en tiempo real (1×)

Al ejecutar el pipeline a velocidad real (`REPLAY_SPEEDUP=1`) sobre el intervalo representativo 66:15–67:15, el tiempo de evento coincide con el reloj local y la marca de agua refleja con exactitud el progreso de los datos. Este escenario demostró la totalidad de la semántica temporal planificada:

Tabla 4. Comportamiento temporal validado en Docker a velocidad real (ejecución del 2026-09-25)

Escenario evaluado	Manifestación observada en almacenamiento
Flujo nominal	Cada intervalo genera emisiones `EARLY` de tamaño creciente y un panel `ON_TIME` (n ≤ 30) al completarse la marca de agua.
`ULT1-b07` (retraso 0–6 min)	La ventana 15:50 se encontraba ausente; tras la reconexión, 31 paneles `LATE` ajustan progresivamente el conteo de 1 a 30 lecturas.
`ULT1-b12` (retraso 20–25 min)	La ventana 16:10 nunca se materializa: sus 30 lecturas superaron el umbral de 15 min de retraso permitido y fueron descartadas silenciosamente por Beam.
Cierre de intervalo	Al expirar el período de latencia, cada ventana materializa un panel `LATE` final con bandera `is_final: true`.

La progresión de paneles se valida de manera reproducible mediante pruebas deterministas con `TestStream` (`tests/test_windows.py`), donde el avance de la marca de agua y la inyección de registros se controlan analíticamente:

Tabla 5. Evolución de paneles bajo control analítico (`tests/test_windows.py`, `TestStream`)

Tipo de panel	Índice	n	Máx.	Evento causante
EARLY	0	1	−80,0 °C	Primera lectura recibida; duplicados redundantes son ignorados.
ON_TIME	1	3	−79,0 °C	La marca de agua supera el fin de ventana con 3 lecturas consolidadas.
LATE	2	4	−78,0 °C	Ingreso de lectura retrasada dentro del margen; actualiza la temperatura máxima.

6 Deduplicación, idempotencia y semántica de entrega

El diseño preliminar consideraba una etapa separada de `GroupByKey` por identificador previa a la combinación. Sin embargo, bajo acumulación continua con redisparos (`early`/`late`), cada emisión retransmite el conjunto acumulado completo, provocando que etapas posteriores dupliquen las sumas. Para evitar sobreestimaciones, la deduplicación se incorporó dentro del acumulador de `TempStatsCombineFn` (`{event_id: temp_c}`), cuyo ciclo coincide con la duración de la ventana más el retraso permitido.

Tabla 6. Garantías semánticas articuladas por etapa de procesamiento

Etapa de flujo	Garantía	Mecanismo técnico	Condición de frontera
Replay → Kafka	Al menos una vez	Productor idempotente (`acks=all`)	Duplicados inyectados por diseño del ensayo
Kafka → Beam	Al menos una vez	Puntos de control Flink cada 30 s	Un reinicio manual del pipeline inicia desde compensaciones confirmadas y descarta ventanas abiertas
Núcleo Beam	Efectivamente una vez por ventana	Deduplicación en acumulador de `CombineFn`	Eventos duplicados posteriores al cierre de ventana se descartan por latencia
Beam → Kafka	Al menos una vez	`WriteToKafka` sin transacciones distribuidas	Posible republicación de paneles tras reinicios no planificados
Destino SQLite	Consistencia final convergente	Operación upsert conservando mayor `pane_index`	Paneles con índice inferior al consolidado se desestiman y auditan en `pane_log`

En conjunto, los tramos que involucran a Kafka son de al menos una vez y la exactitud se recupera por dos vías: la deduplicación por `event_id` dentro de cada ventana y el upsert por `pane_index` en el destino. Por eso no se afirma exactly-once de extremo a extremo, sino un resultado final correcto aunque un pane o un evento se entreguen más de una vez.

7 Verificación empírica y trazabilidad de evidencia

Tabla 7. Estructura de la suite de pruebas unitarias y de integración

Archivo de prueba	Alcance técnico evaluado
`test_contracts.py`	Validación campo a campo, F→C, esquema determinista `event_id`
`test_dataset.py`	Mapeo de nombres en CSV, tiempo transcurrido > 24 h y celdas nulas
`test_producer.py`	Temporizador de replay, retención, ráfagas idénticas y semilla fija
`test_transforms.py`	Estadísticos en `CombineFn`, deduplicación interna y poda de paneles vacíos
`test_windows.py`	`TestStream`: emisiones anticipadas, puntuales, tardías y descartes
`test_sink.py`	Idempotencia en SQLite: mayor índice de panel y auditoría de descartes
`scripts/smoke.py`	Verificación en Docker: tope de lecturas por ventana (31) y DLQ íntegra

Tabla 8. Registro de evidencia experimental recopilada en `docs/evidencia/`: corrida a 20× del 2026-09-24, `smoke.txt` y demo a 1× del 2026-09-25, capturas `vm_*` del 2026-09-29

Archivo de evidencia	Parámetros y resultados verificados
`producer.log`	22 019 eventos; 399 duplicados, 20 ráfagas en `ULT1-b03`, 134 inválidos; balance: 5 487 / 6 615 / 5 512 / 4 405 (ratio 1,50×)
`report_antes_b07.txt` / `_despues_b07.txt`	Estado al liberar `ULT1-b07` y final: 359 a 740 ventanas; ventana 14:30 pasa de n = 14 a 29
`historial_b07_b12.txt`	Paneles por sensor: `ULT1-b12 | 14:50` se actualiza de 15 a 30 lecturas tras la liberación a 20×
`dlq.txt`	134 eventos: 73 por omisión de `temp_c` y 61 por datos no numéricos
`flink-ui.png`	Grafo en Flink: 22 019 consumidos, 21 885 a combinación, 134 a DLQ
`smoke.txt`	Ejecución de `make smoke`: máx. 30 por ventana, suma 2 346 válidos y 59 en DLQ
`tiempo_real_report.txt`, `_producer.log`	Corrida a 1×: 7 373 mensajes, 239 ventanas, 7 119 válidos, 270 tardíos y descarte de `ULT1-b12`
`vm_replay20x_*.png`, `vm_tiempo_real_*.png`	Validación en Debian 13: 77 pruebas OK, balances idénticos y 561 ventanas fuera de rango

8 Límites identificados y consideraciones operativas

- **Sensibilidad temporal**: Al operar con `CreateTime`, la marca de agua queda anclada al reloj físico; a 20× las ventanas se emiten como `EARLY` y `ULT1-b12` corrige en vez de descartarse. La política completa se valida a 1× o con `TestStream`.
- **Distribución térmica**: 558 de 740 ventanas alertan por frío (< -80 °C, mín. $-82{,}9$ °C frente a $-76{,}9$ °C en apertura); en producción conviene separar alertas de sobrecalentamiento (> -60 °C) de métricas de congelación profunda.
- **Entorno Linux y estabilidad**: La validación en Debian 13 requirió UID 1000 en el contenedor, permisos en `/src`, `heartbeat.timeout` de 180 s y `restart: unless-stopped` en TaskManagers para evitar `OutOfMemoryError: Metaspace` ante pausas del anfitrión.
- **Fronteras de emisión y lectura**: Se filtran paneles vacíos emitidos por Flink al cerrar lecturas acotadas; Beam descarta en silencio eventos fuera de latencia (verificado por ausencia de ventana); y la lectura clásica (`use_deprecated_read`) resolvió la pérdida del último lote de la DLQ observada con SDF.
- **Confirmación de offsets**: el lector usa `enable.auto.commit`, que no se coordina con los checkpoints y puede dar por leído algo que sigue en una ventana abierta. Mejora: `commit_offset_in_finalize=True`, que confirma solo al completarse un checkpoint (no probado; sin savepoint igual se pierden las ventanas abiertas).
- **Entorno de laboratorio**: un solo broker con factor de replicación 1; zona horaria del ensayo asumida (America/New_York), que no afecta a las ventanas porque el replay desplaza el tiempo de evento al presente; el tramo 65:00–68:00 no tiene celdas vacías reales, así que la DLQ se demuestra con inválidos inyectados.
- **Mejoras posibles**: tópico `coldchain.alerts.breach.v1` con estado y timers, tópico `too_late` explícito para los descartes, tablero sobre el SQLite y un segundo tipo de evento (O_2/CO_2).

9 Referencias

1. Sun, Z., Ostrowski, K., et al.: Dataset of ultralow temperature refrigeration for COVID-19 vaccine distribution solution. *Scientific Data* 9, 85 (2022). DOI: 10.1038/s41597-022-01167-y (Datos en figshare: 10.6084/m9.figshare.14888121, CC BY 4.0).